Projet OPTICAL : Estimation du flot optique

Présentation du problème

Le **flot optique** (optical flow) est le champ de vecteurs décrivant le mouvement apparent des pixels entre deux images successives d'une vidéo. C'est un outil fondamental en vision par ordinateur pour l'analyse de mouvement, le suivi d'objets, et la reconstruction 3D.
Soit $I(x, y, t)$ l'intensité lumineuse au pixel (x, y) à l'instant t. L'hypothèse de **constance de l'intensité** suppose qu'un point de la scène conserve son intensité au cours du temps :

$$I(x, y, t) = I(x + u, y + v, t + 1) \tag{1}$$

où (u, v) est le déplacement du pixel entre les deux frames.
Un développement de Taylor au premier ordre donne l'**équation du flot optique** :

$$I_x u + I_y v + I_t = 0 \tag{2}$$

où $I_x = \frac{\partial I}{\partial x}$, $I_y = \frac{\partial I}{\partial y}$, $I_t = \frac{\partial I}{\partial t}$ sont les gradients spatio-temporels.
Cette équation unique ne permet pas de déterminer les deux inconnues (u, v) en chaque pixel : c'est le **problème d'ouverture** (aperture problem). Des contraintes supplémentaires sont nécessaires.

Formulation mathématique

Méthode de Horn-Schunck : On ajoute une contrainte de **régularisation spatiale** (lissage du champ de flot) :

$$E(u, v) = \iint_{\Omega} \left[(I_x u + I_y v + I_t)^2 + \alpha \left(|\nabla u|^2 + |\nabla v|^2 \right) \right] dx\, dy \tag{3}$$

où $\alpha > 0$ est le paramètre de régularisation.
Les équations d'Euler-Lagrange associées sont :

$$\begin{aligned} I_x(I_x u + I_y v + I_t) - \alpha \Delta u &= 0 \quad (4) \\ I_y(I_x u + I_y v + I_t) - \alpha \Delta v &= 0 \quad (5) \end{aligned}$$

où $\Delta = \frac{\partial^2}{\partial x^2} + \frac{\partial^2}{\partial y^2}$ est le laplacien.
Méthode de Lucas-Kanade : On suppose le flot constant dans une fenêtre locale $\mathcal{W}$ autour de chaque pixel :

$$\min_{u,v} \sum_{(x_i, y_i) \in \mathcal{W}} w_i \left(I_x(x_i, y_i) u + I_y(x_i, y_i) v + I_t(x_i, y_i) \right)^2 \tag{6}$$

où w_i sont des poids (souvent gaussiens). C'est un problème de moindres carrés linéaires en (u, v).
Approche multi-échelle : Pour gérer les grands déplacements, on utilise une pyramide d'images : on estime d'abord le flot à basse résolution, puis on l'affine aux résolutions supérieures.

Travail demandé

1. Implémenter la méthode de Horn-Schunck. Résoudre les équations d'Euler-Lagrange par un schéma itératif (Gauss-Seidel, Jacobi).
2. Étudier l'influence du paramètre α sur le compromis fidélité aux données / régularité.
3. Comparer avec la méthode de Lucas-Kanade (disponible dans OpenCV).
4. Implémenter une version multi-échelle pour gérer les grands déplacements.
5. Tester sur des vidéos filmées avec smartphone : écoulement d'eau/fumée, objets en mouvement, etc.
6. **Extension :** Remplacer la norme L^2 par une norme L^1 (variation totale) pour mieux préserver les discontinuités.

Annexe : Flot optique avec OpenCV

OpenCV fournit des implémentations de Lucas-Kanade (sparse) et Farnebäck (dense).

Listing 1 – Flot optique avec OpenCV

```
import cv2
import numpy as np

def lucas_kanade_sparse(video_path, max_corners=100):
    """
    Flot optique sparse avec Lucas-Kanade.
    Suit des points d'interet entre frames successives.
    """
    cap = cv2.VideoCapture(video_path)

    # Parametres pour la detection de coins
    feature_params = dict(
        maxCorners=max_corners,
        qualityLevel=0.3,
        minDistance=7,
        blockSize=7
    )

    # Parametres Lucas-Kanade
    lk_params = dict(
        winSize=(15, 15),
        maxLevel=2,
        criteria=(cv2.TERM_CRITERIA_EPS | cv2.TERM_CRITERIA_COUNT, 10, 0.03)
    )

    ret, old_frame = cap.read()
    old_gray = cv2.cvtColor(old_frame, cv2.COLOR_BGR2GRAY)
    p0 = cv2.goodFeaturesToTrack(old_gray, mask=None, **feature_params)

    trajectories = []

    while True:
        ret, frame = cap.read()
        if not ret:
            break
```

```
        frame_gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

        # Calcul du flot optique
        p1, status, err = cv2.calcOpticalFlowPyrLK(
            old_gray, frame_gray, p0, None, **lk_params
        )

        # Selection des bons points
        if p1 is not None:
            good_new = p1[status == 1]
            good_old = p0[status == 1]

            # Stockage des deplacements
            for new, old in zip(good_new, good_old):
                trajectories.append({
                    'old': old.flatten(),
                    'new': new.flatten(),
                    'flow': (new - old).flatten()
                })

        old_gray = frame_gray.copy()
        p0 = good_new.reshape(-1, 1, 2)

    cap.release()
    return trajectories

def farneback_dense(frame1, frame2):
    """
    Flot optique dense avec methode de Farneback.
    Retourne le champ de flot (u, v) pour chaque pixel.
    """
    gray1 = cv2.cvtColor(frame1, cv2.COLOR_BGR2GRAY)
    gray2 = cv2.cvtColor(frame2, cv2.COLOR_BGR2GRAY)

    flow = cv2.calcOpticalFlowFarneback(
        gray1, gray2, None,
        pyr_scale=0.5,   # facteur de reduction pyramide
        levels=3,        # nombre de niveaux
        winsize=15,      # taille fenetre
        iterations=3,    # iterations par niveau
        poly_n=5,        # taille voisinage polynome
        poly_sigma=1.2,  # sigma du gaussien
        flags=0
    )

    # flow[:,:,0] = composante u (horizontale)
    # flow[:,:,1] = composante v (verticale)
    return flow

def visualize_flow(flow):
    """
    Visualise le flot optique en HSV (teinte=direction, saturation=magnitude).
    """
    magnitude, angle = cv2.cartToPolar(flow[:,:,0], flow[:,:,1])
    hsv = np.zeros((*flow.shape[:2], 3), dtype=np.uint8)
    hsv[..., 0] = angle * 180 / np.pi / 2  # Teinte
    hsv[..., 1] = 255                      # Saturation
```

```
    hsv[..., 2] = cv2.normalize(magnitude, None, 0, 255, cv2.NORM_MINMAX)
    rgb = cv2.cvtColor(hsv, cv2.COLOR_HSV2BGR)
    return rgb
```

Ressources PyTorch :

- **Kornia** (`https://github.com/kornia/kornia`) : filtres différentiables pour calculer les gradients d'image
- **RAFT** (`https://github.com/princeton-vl/RAFT`) : méthode state-of-the-art basée sur deep learning

Références

- Wikipedia : Optical flow – `https://en.wikipedia.org/wiki/Optical_flow`
- Horn, B.K.P. & Schunck, B.G. (1981). Determining optical flow. Artificial Intelligence, 17, 185-203.
- Lucas, B.D. & Kanade, T. (1981). An iterative image registration technique with an application to stereo vision.
- OpenCV Optical Flow Tutorial – `https://docs.opencv.org/4.x/d4/dee/tutorial_optical_flow.html`